Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Stack operations, push and pop

Today
Welcome to your next lesson: Stack operations, push and pop.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and how you learn best.

Let’s dive in — you’re making steady progress.

Imagine you have a small stack of comic books on your desk 📚

You can:

add a new comic on top
take the top comic off

In stack terms:

adding on top = push
taking the top off = pop

In Python, we’ll use a list, and the right end is the top:

push → my_stack.append(item)
pop → my_stack.pop()
Quick check, Ron:

If stack = ["A", "B"] (A at the bottom, B on top), and you do:

Python

stack.append("C")
Which letter is now on the top of the stack?

"C" i was just going to ask whats the use in push if we have append also pushing at end

You’re exactly right: append already is “push to the end” in Python 👍

In this course, when we say push, we’re really just giving a stack name to an operation you already know:

Concept word (DS world): push to top of stack
Python code: stack.append(value)

So we don’t invent a new behavior — we just agree:

“For our stack, the top is the right end of the list, and we use append to push there.”

Later, if you see stacks in another language, they might literally have a push() method, but in Python-lists, append plays that role.

Now let’s look at the opposite: pop.

Simple version:

pop = “take the top item off and also give it back to you.”

Example:

Python

```
stack = ["A", "B", "C"]  # C is on top
item = stack.pop()
```

After this:

item is "C"
stack is now ["A", "B"]

Does that behavior make sense: pop both removes and returns the top element?

Yes, that makes sense

Perfect ✸

Let’s make it concrete and very step-by-step with a small example.

We’ll treat the right side of the list as the top of the stack.

Python

```
stack = []          # start empty

stack.append("A")   # push A
stack.append("B")   # push B
stack.append("C")   # push C

top_item = stack.pop()  # pop
```

After this code runs:

What is top_item?
And what does stack look like now?

You don’t need to run it yet — just tell me your guess based on the rules we talked about 👇

"C"
["A", "B"]

Exactly right on both ☑

top_item → "C"

stack → ["A", "B"]

So now you've seen the key difference:

push (append): changes the list, returns None (we usually don't store its result)
pop: changes the list and returns the removed value
Next small step: I want you to try this in the code editor so it sticks.

Do this in the editor (don't paste here):

Create a stack:

stack = []
Push three items with append, for example "red", "green", "blue" (in that order).

Pop once into a variable, like:

top = stack.pop()
print(top) and print(stack) to see both.

Run it, then tell me:

What did top print?
What did stack print?
blue
["red", "green"]

Nice, that's exactly what I'd expect 👌

top → "blue"
stack → ["red", "green"]

So your mental model of push/pop is solid.

One more key idea for stacks:

If you keep popping and the stack becomes empty ([]), then calling stack.pop() again will cause an error.

To guard against that, we usually check:

Python

```
if len(stack) == 0:
    print("Stack is empty")
else:
    value = stack.pop()
    print("Popped:", value)
```

Step by step:

len(stack) == 0 → means the stack has no items.
Only if it's not empty do we call pop().

Does this length-check pattern make sense so far? 🍸

Yes, that makes sense

What do you think the answer is? Pick one below 👇

If you push 'shield' onto ['helmet', 'sword'] and then pop once, what value is returned?

A.

'helmet'

B.

'sword'

C.

'shield'

D.

None

Nice work, Ron — you nailed it 🎯

We start with:

Python

```
stack = ["helmet", "sword"]
stack.append("shield")   # push
returned = stack.pop()   # pop
```

Step by step:

After append("shield"), the stack is ["helmet", "sword", "shield"]
top = "shield" (rightmost item)
pop() removes the top and returns it
returned becomes "shield"
stack becomes ["helmet", "sword"]

So "shield" is exactly the right answer ☑

You've shown you can:

use append as push
use pop to remove and get the top item
and you understand the behavior clearly

Do you feel ready to move on to the next lesson, or do you want one more practice with push/pop first?

I'm ready to move on

Love it. You're moving with real momentum now 🚀

Quick recap of what you just mastered:

You use a Python list as a stack, with the right end as the top.
Push = stack.append(item) → changes the stack, doesn't return a value.
Pop = value = stack.pop() → removes the top item and returns it.
You can safely pop by checking len(stack) == 0 first when needed.
You're ready for the next step in your data structures journey 🔧
When you're back in the Learn tab, just hit Next Lesson and we'll keep going.

If anything about stacks feels fuzzy later, just say "revisit push/pop" and I'll walk through it with you again.

```
stack = []
stack.append("red")
stack.append("green")
stack.append("blue")
if len(stack) != 0:
  top = stack.pop()
```

```
270   print(f'top = {top}')
271   print(f'stack = {stack}')
```